# Pravadh DocSign

Solution Architecture and Usage: Salesforce AppExchange Security Review

| | |
|---|---|
| **Publisher** | Pravadh Labs |
| **Package** | Pravadh DocSign, second-generation managed package, namespace `prvd` |
| **Version under review** | 0.11.0 (04tdN000000SNo5QAG) |
| **Document date** | 29 September 2026 |

### Contents


1. Solution overview
2. Architecture and components
3. Information flows
4. Authentication and authorization
5. Encryption and data protection
6. Data touchpoints
7. Installation and configuration
8. Usage instructions (reviewer walkthrough)


## 1. Solution overview

Pravadh DocSign is a Salesforce-native document generation and electronic signature application. Users generate documents from templates that merge data from any Salesforce record, or upload an existing PDF. They then send the document for signature to one or more recipients. External signers review and sign on a public Salesforce Site page without a Salesforce login. On completion, the signed PDF and a Certificate of Completion are saved as Salesforce Files on the originating record.

> **Everything runs inside the subscriber org.** The package makes no HTTP callouts. It has no Remote Site Settings, Named Credentials, connected apps, external services, external JavaScript or CSS, and no web app or platform outside Salesforce. It stores no Salesforce data outside Salesforce. The only traffic that leaves the org is standard Salesforce email (signing requests and notifications) and the signer's browser session with the Salesforce Site.

## 2. Architecture and components

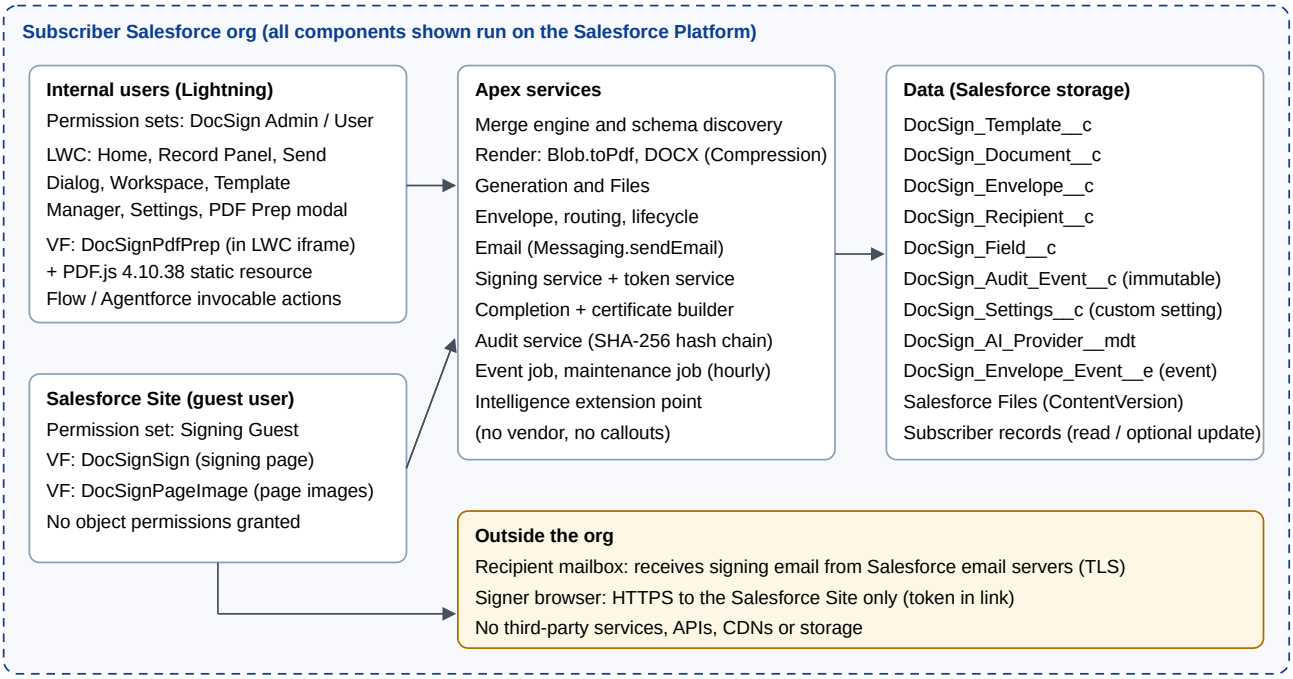



Figure 1. Component view. Arrows show calls; all storage is standard Salesforce storage in the subscriber org.

### 2.1 Components

| Layer | Components |
|---|---|
| Lightning Web Components | `docSignHome`, `docSignRecordPanel`, `docSignSendDialog`, `docSignWorkspace`, `docSignTemplateManager`, `docSignSettings`, `docSignPdfPrep`, `docSignTips`, `docSignPromo`, `docSignAbout`, `docSignStatusBadge`, `docSignUtils` |
| Visualforce pages | `DocSignPdfPrep` (internal PDF field editor, authenticated users only), `DocSignSign` (public signing page), `DocSignPageImage` (public, read-only page images, text/plain) |
| Apex: generation | `DocSignSchemaService`, `DocSignMergeEngine`, `DocSignFormatter`, `DocSignRenderService`, `DocSignDocxRenderer`, `DocSignGenerationService`, `DocSignFileService`, `DocSignPdfLayout` |
| Apex: envelopes and signing | `DocSignEnvelopeService`, `DocSignRoutingService`, `DocSignLifecycle`, `DocSignEmailService`, `DocSignReminderSender`, `DocSignSigningService`, `DocSignTokenService`, `DocSignSigningController`, `DocSignPageImageController` |
| Apex: completion, audit and async | `DocSignCompletionService`, `DocSignCertificateBuilder`, `DocSignAuditService`, `DocSignAuditGuard`, `DocSignEvents`, `DocSignEventHandler`, `DocSignEventJob` (Queueable), `DocSignMaintenanceJob` (Schedulable, hourly, optional) |
| Apex: UI controllers | `DocSignHomeController`, `DocSignRecordController`, `DocSignWorkspaceController`, `DocSignTemplateController`, `DocSignPdfPrepController`, `DocSignEmailHealth` |

| | |
|---|---|
| Global API (subscriber use) | Flow invocable actions: Generate Document, Send for Signature, Get Signature Status, Resend Envelope, Cancel Envelope (`DocSignRecipientInput` Apex-defined type). Agentforce-ready actions (`DocSignAgentActions`, `DocSignMissingDataAction`, `DocSignSignatureSummaryAction`). `DocSignIntelligence.Provider`: an interface subscribers may implement; registered through `DocSign_AI_Provider__mdt`. The package ships only a built-in heuristic provider, with no AI vendor and no callouts. All global classes are `with sharing` and run as the calling user. |
| Triggers | `DocSignEnvelopeTrigger` (delete protection), `DocSignAuditEventTrigger` (audit immutability), `DocSignEnvelopeEventTrigger` (platform-event subscriber) |
| Static resources | `DocSignPdfEngine` (PDF.js 4.10.38, Apache-2.0, plus the package's own detection and editor scripts), logo and wordmark images |
| Security metadata | Permission sets *Pravadh DocSign Admin*, *Pravadh DocSign User*, *Pravadh DocSign Signing Guest* (only the two public pages and their two controller classes; no object or field permissions). Custom permission `Pravadh_DocSign_Admin`. |

## 3. Information flows

### 3.1 Template document generation (internal user)

1. The user opens a record and picks a template in the *Pravadh DocSign* record component (LWC).
2. The LWC calls an `@AuraEnabled` method. `DocSignMergeEngine` reads the record and related records with `WITH USER_MODE` SOQL, checking the user's access at each relationship hop. Every merged value is HTML/XML-escaped.
3. `DocSignRenderService` renders HTML to PDF with `Blob.toPdf`, or `DocSignDocxRenderer` renders a DOCX with the Apex Compression API. Images are referenced only as `/sfc/servlet.shepherd/version/download/068…` URLs of Files in the same org (validated ID pattern, never external URLs).
4. The output is saved as a ContentVersion linked to the record, and a `DocSign_Document__c` record is created (user-mode DML). An audit event is written.

### 3.2 Uploaded PDF preparation (internal user)

1. The user uploads a PDF on a record (standard `lightning-file-upload`). Server-side checks cover file type, the `%PDF-` signature, size (50 MB or less) and page count (100 or less).
2. The `docSignPdfPrep` LWC opens the `DocSignPdfPrep` Visualforce page in an iframe, passing the document id, its own origin and a one-time 128-bit nonce.
3. The page downloads the PDF in the browser from the org's own REST API (`GET /services/data/v67.0/sobjects/ContentVersion/{id}/VersionData`), same origin, with the user's session in the `Authorization` header (see section 4.4). PDF.js renders pages locally with `isEvalSupported:false` and `enableXfa:false`. Embedded PDF JavaScript is never run.
4. Detection runs in the browser (blank lines, checkboxes, form widgets, signer blocks). The user reviews fields and assigns them to signers.
5. The page sends Apex calls to the parent LWC with `postMessage`, using the exact parent origin and the nonce. The LWC accepts only messages from Salesforce origins with the matching nonce. It relays only five allow-listed `@AuraEnabled` methods (`getContext`, `savePage`, `saveLayout`, `getPdfData`, `saveDraft`) and always substitutes the document id it opened.
6. `DocSignPdfPrepController` (`with sharing`, user mode) checks edit access to the document, validates each JPEG page image (1.2 MB or less, JPEG signature) and the field layout (page bounds, field types, 20 roles or fewer, 600 fields or fewer), and stores page images as Files and the layout on the document.

### 3.3 Sending for signature

1. The user picks recipients (name, email, signing order, optional access code, expiry) in the send dialog, or a Flow calls *Send for Signature*.
2. `DocSignEnvelopeService` creates the envelope, recipient and field records in user mode.
3. For each active recipient, `DocSignTokenService` generates a 256-bit random token (`Crypto.generateAesKey(256)`, URL-safe Base64). Only its SHA-256 hash is stored, in `Token_Hash__c`. An optional access code is stored only as a SHA-256 hash salted with the recipient id.
4. `DocSignEmailService` sends the signing email with `Messaging.sendEmail`, from an org-wide email address if one is configured. The email contains the Site link `…/DocSignSign?t=<token>`. The raw token exists only in that email and is not stored or logged.
5. An audit event records the send. The envelope status is *Sent*.

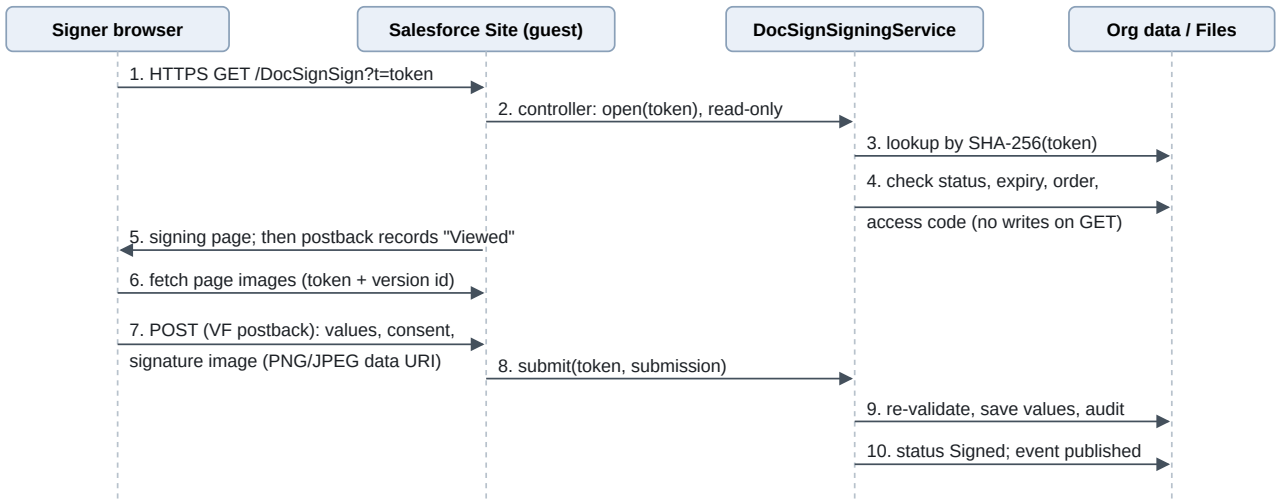



Figure 2. External signing sequence. Every request re-validates the token; the guest user has no object permissions.

### 3.4 External signing

1. The signer opens the link over HTTPS. The `DocSignSign` controller (read-only on page load) checks the token format, looks up the recipient by `SHA-256(token)`, then checks envelope status, expiry and routing order, plus the access code if one is set (5 wrong attempts lock the link). Once the page has rendered, it makes one `apex:actionFunction` postback (view-state CSRF protection) that records *Viewed* and an audit event. A plain GET of the link changes nothing.
2. The page shows the document in an iframe sandboxed without `allow-scripts`. For uploaded PDFs it loads each page image from `DocSignPageImage?t=…&v=068…`. Each request re-validates the token and envelope state and serves an image only if it is registered on that envelope's document.
3. The signer fills only their own fields, gives electronic-records consent, and draws or types a signature. The submission is a Visualforce form postback (view-state CSRF protection). The server re-validates everything: token, state, required fields, field ownership, lengths, and that the signature is a PNG/JPEG data URI. Name, email and date values are filled in by the system, not taken from the signer.
4. Recipient, field and audit records are updated in system mode, limited to that signer's own records. IP address and user agent are recorded for the certificate. A `DocSign_Envelope_Event__e` platform event is published. The link then only shows "already signed" until the envelope completes, when every token hash is cleared.
5. Declining follows the same checks, voids the other links and notifies the sender.

### 3.5 Routing and completion (asynchronous)

1. `DocSignEnvelopeEventTrigger` receives the platform event and enqueues `DocSignEventJob`.

2. If signers remain, the next routing group receives new tokens and emails. When all have signed, `DocSignCompletionService` checks every recipient has signed and that the envelope isn't finalized yet (`Finalized__c`, so it runs only once). It renders the signed PDF and the Certificate of Completion (signers, IP addresses, user agents, timestamps, SHA-256 hashes of the original and final documents, and the audit hash chain).
3. Both PDFs are saved as Files on the source record and the DocSign document. If the admin has mapped one, a field on the source record is updated. A completion email goes to every signer and the sender, with the signed PDF and certificate attached (when both together are under 9 MB).
4. The optional hourly `DocSignMaintenanceJob` expires overdue envelopes (clearing their tokens) and sends reminders, which replace the tokens.

## 4. Authentication and authorization

| Actor | Authentication | Authorization |
|---|---|---|
| Internal user | Standard Salesforce login (org's SSO/MFA policies). | DocSign Admin or User permission set. All user-facing Apex is `with sharing`, SOQL `WITH USER_MODE`, DML `as user`. Merge paths are checked per relationship hop. Settings changes need the `Pravadh_DocSign_Admin` custom permission. PDF editing checks `UserRecordAccess.HasEditAccess`, and sent documents are locked. |
| External signer | Possession of the 256-bit token from the signing email; optionally an access code shared out-of-band. | Site guest user with only the Signing Guest permission set (no object permissions). Guest Apex runs in system mode only after token validation, and only on that signer's recipient and field records and that envelope's page images. The envelope's status, expiry and routing order are checked on every request. After 5 wrong access codes the link locks. |
| Automated processes | Platform event subscriber (Automated Process user, or an integration user set by the admin) and the scheduled job's running user. | Triggered only by the package's own event or schedule. Completion is idempotent and re-verifies signer status before finalizing. |
| Flow / Agentforce | The running user of the flow or agent. | Global invocable actions are `with sharing` and delegate to user-mode services. |

### 4.1 Why some classes run in system mode

External signers are Site guest users, who own no records and have no object permissions (Salesforce secure guest sharing). The public page controllers (`DocSignSigningController`, `DocSignPageImageController`) are `with sharing` and do no SOQL or DML themselves. The services they call, plus the audit, routing, completion and event classes, are `without sharing` and use `AccessLevel.SYSTEM_MODE`. None of them can be reached without one of: (a) a validated signer token, (b) a user-mode operation on the same record just before, or (c) the package's own platform event or scheduled job. Audit events can be read by users but not created, updated or deleted by them.

### 4.2 Signing tokens

- Generated with `Crypto.generateAesKey(256)`: 256 bits from Salesforce's cryptographic random source.
- Stored only as `SHA-256("pravadh-docsign-token:" + token)`. Someone who can read the database can't reconstruct a signing link.
- Replaced on resend and reminder. Cleared for all recipients when the envelope completes, is declined, is cancelled or expires. A signer who has already signed can only see an "already signed" message.

- Signing pages send `<meta name="referrer" content="no-referrer">` and `robots noindex, nofollow`, so the token doesn't leak through the Referer header or search indexing.

### 4.3 Cross-frame messaging (PDF editor)

The Visualforce editor runs on the org's Visualforce domain, inside the LWC modal. Visualforce remoting is not available on header-less pages, so the editor asks the parent LWC to call Apex. The editor posts only to the exact parent origin it was given; the controller validates that origin as an https Salesforce domain. The LWC accepts only messages from Salesforce origins carrying its one-time nonce. It relays only five allow-listed `@AuraEnabled` methods of the `with sharing` controller and always substitutes its own document id. The relay therefore gives the user nothing they couldn't already call.

### 4.4 Session ID use in `DocSignPdfPrep`

The editor reads `$Api.Session_ID` only to download the uploaded PDF from the org's own REST API on the same Visualforce origin (`Authorization: Bearer`). Salesforce's `/sfc/servlet.shepherd` file URL redirects to the `*.file.force.com` content domain, and CORS blocks that download. The session ID is kept in memory only. It is never placed in a URL, stored, logged or sent to another host, and the request honours the user's own file access. Without API access, the editor falls back to an Apex read (2 MB or less) or a local file picker, where the chosen file must match the uploaded size.

## 5. Encryption and data protection

| Path | Protection |
|---|---|
| Internal user ↔ Salesforce | HTTPS (TLS 1.2+) enforced by the Salesforce Platform, including the editor's REST call and the LWC-to-Apex calls. |
| Signer browser ↔ Salesforce Site | HTTPS only. The setup guide requires the Site to *Require Secure Connections (HTTPS)* and recommends clickjack protection "Allow framing by the same origin only". |
| Salesforce ↔ recipient mail server | Standard Salesforce email delivery, which uses TLS whenever the receiving server supports it (the org's TLS setting). Customers can enforce TLS and sign mail with DKIM for their domain; DocSign Settings shows DKIM status and links to setup. |
| Data at rest | All data stays in Salesforce storage (records and Files) under Salesforce's infrastructure security. The package adds no separate data store. Secrets are never stored in reversible form: signing tokens and access codes are one-way SHA-256 hashes. |
| Integrity | SHA-256 hashes of the original and final documents. The audit trail is a SHA-256 hash chain (each event includes the previous hash), checked on every workspace load and printed on the Certificate of Completion. |
| Output encoding | All merged values are HTML/XML-escaped (the `raw` filter is opt-in for admin-authored rich text). Signer-supplied values are escaped before rendering. The document preview uses a sandboxed iframe without scripts. |

## 6. Data touchpoints

| Data | Where stored | Who can access / notes |
|---|---|---|
| Subscriber record data used in templates | Not copied, except into the generated document | Read in user mode with per-hop access checks. It appears only in the generated File. |
| Templates (HTML or DOCX, settings) | `DocSign_Template__c`, Files | DocSign Admin creates and edits; Users read. |

| | | |
|---|---|---|
| Generated, uploaded, signed PDFs and certificates | Salesforce Files linked to the source record and the DocSign document | Standard Files sharing, inherited from the linked records. |
| PDF page images and field layout | Files; `DocSign_Document__c.Page_Images__c` and `Layout_Json__c` | Document editors; served to signers only through the token-checked page-image endpoint. |
| Recipient name, email, routing order | `DocSign_Recipient__c` | Sender and users with access to the envelope. The signer sees only their own details and the names of other signers. |
| Signer IP address and user agent | `DocSign_Recipient__c`, `DocSign_Audit_Event__c` | Collected for evidence and shown on the Certificate of Completion. |
| Signature and initials images | `Signature_Image__c` and Files (`Signature_File_Id__c`, `Initials_File_Id__c`) | Visual marks rendered into the signed PDF. Not credentials. |
| Field values entered by signers | `DocSign_Field__c.Value__c` | Validated and length-limited; merged into the signed PDF. |
| Token and access-code hashes | `DocSign_Recipient__c.Token_Hash__c`, `Access_Code_Hash__c` | One-way SHA-256 only. Cleared when the link ends. |
| Audit trail | `DocSign_Audit_Event__c` | Read-only for users. Insert only through the package service; update and delete blocked. |
| Settings | `DocSign_Settings__c` (hierarchy custom setting) | Branding, Site URL, sender address, reminder options. Written only by users with the admin custom permission. No secrets. |
| Email | Salesforce email; the org's Email Logs | Signing requests contain the signer's name, envelope title, the sender's message and the signing link, with no attachments. Completion emails to participants attach the signed PDF and Certificate of Completion. Decline notices go to the sender. |
| Browser storage | Signer's or user's browser | Internal users only: the in-app tips and promo card dismissal flags (`localStorage`). No record data or tokens. The signing page uses no browser storage. |

**Not collected:** the package doesn't send data to Pravadh Labs or any third party, and has no telemetry, analytics or tracking. The in-app *About* and promo cards contain links to pravadhlabs.com that open only when a user clicks them (`rel="noopener noreferrer"`). They send no data.

## 7. Installation and configuration

1. Install the package (Admins only or specific profiles). Assign **Pravadh DocSign Admin** to administrators and **Pravadh DocSign User** to users who generate and send documents.
2. Add the **Pravadh DocSign** component to record pages in Lightning App Builder. The Pravadh DocSign app includes Home, Templates, Documents, Envelopes and Settings tabs.
3. For external signers:
   a. Setup → Sites: create a Site, enable *Require Secure Connections (HTTPS)*, and add the Visualforce pages `prvd__DocSignSign` and `prvd__DocSignPageImage`.

   b. Assign the **Pravadh DocSign Signing Guest** permission set to the Site's guest user. Don't grant the guest any other object access.
   c. Paste the Site URL into DocSign → Settings → Signing Site URL.
4. Email: Setup → Deliverability → Access level *All email*. Optionally set an org-wide email address and DKIM for your domain. DocSign Settings has a *Send test email* button and an email health check.
5. Optional: in Settings, turn on the hourly maintenance job for reminders and expiry. Optionally set a platform-event subscriber user.

## 8. Usage instructions (reviewer walkthrough)

The test org provided with this submission is configured as above, with sample templates and records. Use your own email address as the signer so you receive the signing links.

### 8.1 Generate a document from a template

1. Open a sample Account, Contact or Opportunity. In the **Pravadh DocSign** panel, select a sample template and click **Generate Document**.
2. The PDF is created and appears in the panel and under the record's Files. Open it to check the merged values.

### 8.2 Send for signature (two signers, sequential)

1. Click **Send for Signature**. Add signer 1 and signer 2 using email addresses you can access. Optionally set an access code for signer 2. Click **Send**.
2. Open the email for signer 1 and click the link. The signing page opens on the Salesforce Site without a login.
3. Fill the highlighted fields, accept the electronic-records consent, draw or type a signature (**Adopt and Sign**) and click **Finish**.
4. Signer 2 then receives the email. Open the link, enter the access code if you set one, and sign.
5. After a short asynchronous step, the record's Files show the **signed PDF** and the **Certificate of Completion**. The envelope status is *Completed*.

### 8.3 Sign an uploaded PDF

1. On a record, choose **Upload a PDF for signature** and upload any PDF (100 pages or fewer).
2. The **Prepare** editor opens with a progress bar and suggested fields. Adjust fields, assign them to signers (the legend shows who fills what), fill any "fill now" values, then **Save & continue** and **Send**.
3. Sign as in 8.2. Page images load one page at a time on the signing page.

### 8.4 Other scenarios to test

- **Decline:** open a signing link and choose Decline with a reason. The envelope becomes Declined and the other links stop working.
- **Replaced link:** resend an envelope, then open the old link. It is rejected as replaced.
- **Tampered or random token:** `/DocSignSign?t=abc…` shows "invalid link". `DocSignPageImage` with a valid token but another document's image id returns an empty response.
- **Access code lockout:** five wrong codes lock the link.
- **Audit trail:** open the DocSign Document record. The workspace shows the timeline and a verified hash chain. Try editing or deleting an audit event as an admin; it's blocked.
- **Flow:** the invocable actions *Generate Document* and *Send for Signature* are available in Flow Builder under "Pravadh DocSign".

### Related documentation (attached)

- *DocSign-Scanner-Findings-and-Justifications.pdf*: every Checkmarx (56) and Code Analyzer finding with its justification.
- Checkmarx Source Code Scanner report (Partner Security Portal) and Salesforce Code Analyzer v5 reports.